

UniApp tabBar底部菜单配置及iconfont图标使用全指南

在UniApp跨平台开发中，tabBar是实现多模块快速切换的核心组件，而iconfont图标以其体积小、可定制性强的优势，成为tabBar图标的优选方案。本文将从tabBar基础配置、iconfont图标引入、关联配置到多端适配，完整呈现底部菜单与图标结合的实现流程，解决图标模糊、适配异常等常见问题。

一、tabBar底部菜单基础配置：核心结构与规则

tabBar通过pages.json中的 `tabBar` 节点配置，核心是 `list` 数组，每个数组项对应一个底部菜单选项，包含页面关联、文字、图标等关键信息。配置前需明确基础规则，避免路由冲突与样式异常。

1. 基础配置结构

一个标准的tabBar配置需包含菜单选项列表、颜色、背景等基础属性，示例如下：

Code block

```
1  {
2    "pages": [
3      "pages/index/index",    // 菜单1关联页面（必须在pages数组中）
4      "pages/category/category", // 菜单2关联页面
5      "pages/cart/cart",      // 菜单3关联页面
6      "pages/user/user"      // 菜单4关联页面
7    ],
8    "tabBar": {
9      "color": "#666666",      // 未选中菜单文字颜色
10     "selectedColor": "#1890FF", // 选中菜单文字颜色
11     "backgroundColor": "#FFFFFF", // 菜单背景色
12     "borderStyle": "black",    // 菜单上边框颜色（仅支持black/white）
13     "height": "50px",         // 菜单高度（默认50px，可调整）
14     "list": [                 // 底部菜单选项列表（2-5项）
15       {
16         "pagePath": "pages/index/index", // 关联页面路径（与pages数组一致）
17         "text": "首页",                // 菜单文字
18         "iconPath": "static/tab/index.png", // 未选中图标
19         "selectedIconPath": "static/tab/index-active.png" // 选中图标
20       },
21       // 其他菜单选项...
22     ]
23   }
```

2. 核心配置规则

- **页面关联规则：** `pagePath` 必须与 `pages` 数组中的页面路径完全一致，否则点击菜单无法跳转；关联页面需为非分包页面（分包页面不可作为tab页）。
- **菜单数量限制：** 微信小程序端 `list` 数组长度需在2-5之间，App端无严格限制，但建议不超过5项以保证体验。
- **图标基础要求：** 默认支持本地图片（PNG/JPG），若使用iconfont需通过特殊配置实现；图标建议为正方形，避免拉伸变形。
- **样式优先级：** tabBar样式优先级高于全局样式，且不继承 `globalStyle` 的配置，需单独定义。

二、iconfont图标引入：从下载到项目集成

iconfont（阿里巴巴矢量图标库）是免费图标资源平台，使用前需完成图标挑选、下载，再集成到UniApp项目中。相比本地图片，iconfont图标支持颜色动态修改、缩放不失真，更适配tabBar的选中/未选中状态切换。

1. 步骤1：从iconfont挑选并下载图标

1. 访问[iconfont官网](#)，注册并登录，创建“项目”（建议按项目命名，便于管理）。
2. 搜索目标图标（如“首页”“分类”），点击“添加入库”，完成后进入“购物车”批量添加到已创建的项目中。
3. 进入项目页面，点击“下载至本地”，选择“Font class”格式（UniApp推荐使用，兼容性好），下载压缩包。

2. 步骤2：项目集成iconfont资源

将下载的iconfont文件解压后，按以下步骤集成到UniApp项目：

2.1 放置资源文件

在项目 `static` 目录下创建 `iconfont` 文件夹，将解压后的 `iconfont.css`、`iconfont.ttf`、`iconfont.woff` 等文件复制到该目录（核心文件为.css和字体文件）。

项目目录结构示例：

Code block

```
1  src/
2  |  └─ static/
3  |    └─ iconfont/
4  |        └─ iconfont.css
```

```
5 | iconfont.ttf
6 | iconfont.woff
7 | pages.json
```

2.2 修改iconfont.css路径（关键步骤）

下载的 `iconfont.css` 中，字体文件路径为本地绝对路径，需改为UniApp支持的相对路径，否则图标无法显示。

修改前（默认路径）：

Code block

```
1 @font-face {
2   font-family: "iconfont"; /* Project id 38xxx */
3   src: url('iconfont.ttf?t=1690xxx') format('truetype');
4 }
```

修改后（相对路径，适配UniApp）：

Code block

```
1 @font-face {
2   font-family: "iconfont"; /* 保持原项目ID */
3   src: url('/static/iconfont/iconfont.ttf?t=1690xxx') format('truetype');
4   /* 若有多个字体文件（woff/woff2），依次修改路径 */
5 }
```

2.3 全局引入iconfont样式

在 `App.vue` 的样式中引入 `iconfont.css`，实现全局可用：

Code block

```
1 <script>
2   export default {
3     onLaunch() {
4       // 应用启动逻辑
5     }
6   }
7 </script>
8
9 <style>
10  /* 全局引入iconfont样式 */
11  @import "/static/iconfont/iconfont.css";
12 </style>
```

三、tabBar关联iconfont图标：两种实现方案

UniApp的tabBar默认不直接支持iconfont图标（原生tabBar仅支持图片路径），需通过“自定义tabBar”或“利用原生组件模拟”两种方案实现。其中自定义tabBar灵活性更高，是主流选择。

方案1：自定义tabBar（推荐，多端适配好）

通过UniApp的“自定义tabBar”功能，用组件方式实现底部菜单，直接在组件中使用iconfont图标，支持颜色、大小等动态控制。

1. 步骤1：开启自定义tabBar配置

在pages.json中添加 `"custom": true` 开启自定义tabBar，并保留基础配置（用于指定tab页）：

Code block

```
1  {
2    "pages": [
3      "pages/index/index",
4      "pages/category/category",
5      "pages/cart/cart",
6      "pages/user/user"
7    ],
8    "tabBar": {
9      "custom": true, // 开启自定义tabBar
10     "color": "#666666",
11     "selectedColor": "#1890FF",
12     "list": [
13       { "pagePath": "pages/index/index", "text": "首页" },
14       { "pagePath": "pages/category/category", "text": "分类" },
15       { "pagePath": "pages/cart/cart", "text": "购物车" },
16       { "pagePath": "pages/user/user", "text": "我的" }
17     ]
18   }
19 }
```

2. 步骤2：创建自定义tabBar组件

在项目根目录创建 `custom-tab-bar` 文件夹，新建 `index.vue` 组件，内容如下（核心是用iconfont图标和路由跳转实现菜单功能）：

Code block

```
1  <template>
2    <view class="custom-tab-bar">
```

```
3     <view
4       class="tab-item"
5       v-for="(item, index) in tabList"
6       :key="index"
7       @click="switchTab(item.pagePath)"
8     >
9
10    <text
11      class="iconfont tab-icon"
12      :class="{ 'tab-icon-active': currentTab === index }"
13    >{{ item.icon }}</text>
14
15    <text class="tab-text" :class="{ 'tab-text-active': currentTab === index
16  }">{{ item.text }}</text>
17  </view>
18 </view>
19 </template>
20 <script>
21 export default {
22   data() {
23     return {
24       currentTab: 0, // 当前选中的菜单索引
25       // 菜单列表: icon为iconfont的图标代码 (从iconfont.css获取)
26       tabList: [
27         {
28           pagePath: "pages/index/index",
29           text: "首页",
30           icon: "\ue602" // 图标代码 (替换为自己的iconfont代码)
31         },
32         {
33           pagePath: "pages/category/category",
34           text: "分类",
35           icon: "\ue603"
36         },
37         {
38           pagePath: "pages/cart/cart",
39           text: "购物车",
40           icon: "\ue604"
41         },
42         {
43           pagePath: "pages/user/user",
44           text: "我的",
45           icon: "\ue605"
46         }
47       ]
48     };
49   }
50 }
```

```

49     },
50     methods: {
51         // 切换tab页（必须用switchTab方法）
52         switchTab(pagePath) {
53             uni.switchTab({
54                 url: "/" + pagePath,
55                 success: () => {
56                     // 切换后更新当前选中索引
57                     this.currentTab = this.tabList.findIndex(item => item.pagePath ===
pagePath);
58                 }
59             });
60         },
61     },
62     // 页面显示时同步选中状态（解决返回时状态异常）
63     onShow() {
64         const pages = getCurrentPages();
65         const currentPage = pages[pages.length - 1];
66         this.currentTab = this.tabList.findIndex(item => item.pagePath ===
currentPage.route);
67     }
68 };
69 </script>
70
71 <style scoped>
72 .custom-tab-bar {
73     display: flex;
74     justify-content: space-around;
75     align-items: center;
76     height: 50px;
77     background-color: #fff;
78     border-top: 1px solid #eee;
79     position: fixed;
80     bottom: 0;
81     left: 0;
82     width: 100%;
83     z-index: 999;
84 }
85 .tab-item {

```

3. 步骤3：获取iconfont图标代码

图标代码可从 `iconfont.css` 中获取，每个图标对应一个 `.icon-xxx:before` 选择器，其 `content` 属性值即为图标代码：

Code block

```
1  /* iconfont.css中的图标代码示例 */
2  .icon-shouye:before {
3      content: "\ue602"; /* 这就是首页图标的代码，复制到tabList中 */
4  }
5  .icon-fenlei:before {
6      content: "\ue603"; /* 分类图标代码 */
7  }
```

4. 步骤4：tab页适配底部间距

自定义tabBar固定在底部，会遮挡页面内容，需在所有tab页的根容器添加底部内边距：

Code block

```
1  <template>
2      <view class="index-page">
3
4      </view>
5  </template>
6
7  <style scoped>
8      .index-page {
9          padding-bottom: 50px; /* 与tabBar高度一致 */
10     }
11 </style>
```

方案2：原生tabBar+iconfont转图片（兼容旧项目）

若不想修改为自定义tabBar，可将iconfont图标转为图片（PNG格式），再按原生tabBar的图标路径配置。适合需快速适配的旧项目，但不支持动态修改颜色。

1. 转换方法

1. 在iconfont项目页面，选中图标，点击“下载”旁的下拉箭头，选择“PNG下载”。
2. 设置图片尺寸（建议40x40px）和颜色（未选中用#666，选中用#1890FF），分别下载两种状态的图片。

2. 原生tabBar配置

将转换后的图片放入 `static/tab` 目录，在pages.json中配置路径：

Code block

```
1  {
2      "tabBar": {
3          "list": [
```

```
4      {
5          "pagePath": "pages/index/index",
6          "text": "首页",
7          "iconPath": "static/tab/index.png", // 未选中图片 (iconfont转PNG)
8          "selectedIconPath": "static/tab/index-active.png" // 选中图片
9      }
10    ]
11  }
12 }
```

四、关键问题与多端适配技巧

1. 常见问题解决方案

- **iconfont图标不显示：**

检查 `iconfont.css` 中字体文件路径是否正确（必须是绝对路径 `/static/...`）。

- 确认图标代码是否正确（从css文件复制，避免手动输入错误）。
- 检查是否在App.vue中全局引入了iconfont.css。

自定义tabBar切换后状态异常：在 `onShow` 生命周期中同步当前页面路由与选中索引，确保返回页面时状态正确。

小程序端tabBar图标模糊：若使用图片方案，需提供2倍/3倍图（如iPhone高清屏），或直接用iconfont矢量图标避免模糊。

2. 多端适配要点

- **微信小程序：**自定义tabBar需符合小程序规范，`custom-tab-bar` 文件夹必须放在项目根目录，不可嵌套；图标字体文件需小于40KB（超过需分包）。
- **App端：**支持更大的tabBar高度（如55px），可通过媒体查询适配不同屏幕尺寸；沉浸式导航时需调整tabBar的top值，避免与状态栏重叠。
- **H5端：**自定义tabBar的 `position: fixed` 可能与页面滚动冲突，可添加 `box-sizing: border-box` 优化布局。

3. 优化技巧

- **图标大小统一：**在自定义tabBar中给 `.tab-icon` 设置固定字体大小（如22px），确保所有图标视觉一致。
- **点击反馈：**给tab-item添加 `active-class` 或点击时的样式变化（如opacity: 0.8），提升交互体验。

- **动态修改tabBar：**若需根据用户身份（如普通用户/商家）显示不同菜单，可在自定义tabBar的 `onShow` 中动态修改 `tabList` 数据。

五、总结

tabBar底部菜单与iconfont图标的结合，核心是通过“自定义tabBar”实现灵活配置，步骤可概括为：iconfont图标下载与项目集成→pages.json开启自定义配置→创建tabBar组件并关联图标与路由→适配页面间距与多端差异。相比传统图片图标，iconfont显著提升了样式可定制性与适配效率，而自定义tabBar则解决了原生tabBar的功能局限，是UniApp多端开发的最优实践。实际开发中需重点关注图标路径配置、选中状态同步及多端适配细节，确保底部菜单的稳定性与交互体验。

（注：文档部分内容可能由 AI 生成）